

PROYECTO FINAL

Robot pinza

Nombre: Geronimo Martinez Avila

Fecha: 18/06/2026

Objetivo del proyecto

Este proyecto fue inspirado por mi abuela que no se puede mover mucho y algunas veces no alcanza objetos en su habitación y quería ayudarla cuando no pueda estar en la casa.

Procedimiento:

Nota: Este informe asume que ya tienes instalado Jammy Jellyfish (Linux Ubuntu), Humble (ROS 2), RViz y Gazebo Fortress (Gazebo).

Nav2 y humble:

Preparación: instalación de paquetes

Instalamos el Cartographer:

```
sudo apt install ros-humble-cartographer ros-humble-cartographer-ros
```

Qué hace: Instala Cartographer, el paquete que permite hacer SLAM, es decir, construir un mapa del entorno mientras el robot se mueve.

- `sudo` → ejecuta el comando con permisos de administrador
- `apt install` → instala paquetes en Ubuntu
- `ros-humble-cartographer` → el paquete de Cartographer para ROS 2 Humble
- `ros-humble-cartographer-ros` → la integración de Cartographer con ROS (nodos y mensajes)

Instalar los paquetes base y de simulación de TurtleBot3:

```
sudo apt install ros-humble-turtlebot3 ros-humble-turtlebot3-simulations
```

Qué hace: Instala los paquetes base del TurtleBot3 y los de simulación (mundos y modelos para Gazebo).

- `ros-humble-turtlebot3` → paquetes base del robot: controladores, descripción y teleoperación
- `ros-humble-turtlebot3-simulations` → mundos y modelos para simular el robot dentro de Gazebo

1. Iniciar la simulación (Gazebo)

► Terminal 1

En una terminal, lanza el mundo donde el robot se encuentra en la casa (house):

1.1 Definir el modelo del robot

```
export TURTLEBOT3_MODEL=waffle_pi
```

Qué hace: Le dice a ROS qué modelo de TurtleBot3 vas a usar. Sin esto, el simulador no sabe qué robot cargar.

- `export` → crea una variable de entorno visible para los comandos siguientes
- `TURTLEBOT3_MODEL` → el nombre de la variable que ROS lee para saber el modelo

- `waffle_pi` → el modelo elegido (el que lleva cámara y brazo)

1.2 Abrir Gazebo con el mundo de la casa

```
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

Qué hace: Abre Gazebo con el mundo “casa” y el robot colocado dentro de él.

- `ros2 launch` → arranca varios nodos a la vez desde un archivo de lanzamiento
- `turtlebot3_gazebo` → el paquete que contiene las simulaciones de Gazebo
- `turtlebot3_house.launch.py` → el archivo que carga específicamente el mundo de la casa

2. Iniciar Nav2 con tu mapa específico

► Terminal 2

En una segunda terminal lanzamos el stack de navegación. Aquí especificamos la ruta hacia tu archivo .yaml (asumiendo que está en la carpeta Descargas) y forzamos el uso del tiempo de simulación (`use_sim_time:=True`).

2.1 Definir el modelo (de nuevo)

```
export TURTLEBOT3_MODEL=waffle_pi
```

Qué hace: Vuelve a definir el modelo porque cada terminal es independiente y no hereda la variable de las otras.

2.2 Lanzar Nav2 con el mapa de la casa

```
ros2 launch nav2_bringup bringup_launch.py map:=/home/geronimo/Descargas/map_house.yaml  
use_sim_time:=True
```

Qué hace: Arranca todo el sistema de navegación (Nav2), cargando tu mapa de la casa y usando el reloj de la simulación.

- `ros2 launch nav2_bringup bringup_launch.py` → lanza el stack de navegación Nav2
- `map:=/home/geronimo/Descargas/map_house.yaml` → le indica la ruta del mapa que debe usar
- `use_sim_time:=True` → usa el reloj de la simulación, no el del PC (clave para sincronizar todo en Gazebo)

3. Abrir RViz

► Terminal 3

3.1 Abrir RViz

```
ros2 launch nav2_bringup rviz_launch.py
```

Qué hace: Abre RViz, la ventana donde ves el mapa, el robot y puedes darle metas de navegación.

- `ros2 launch nav2_bringup rviz_launch.py` → abre RViz ya configurado para trabajar con Nav2

4. Abrir el teleoperador

► Terminal 4

4.1 Abrir el control por teclado

```
export TURTLEBOT3_MODEL=waffle_pi && ros2 run turtlebot3_teleop teleop_keyboard
```

Qué hace: Abre el control por teclado para mover el robot manualmente por la casa.

- `ros2 run` → ejecuta un solo nodo (a diferencia de launch, que arranca varios)

- `turtlebot3_teleop` → el paquete de teleoperación
- `teleop_keyboard` → el nodo que lee el teclado y mueve el robot

Manipulation

1. Simulación

► Terminal 1 — preparación e inicio

1.1 Crear el workspace

```
mkdir -p ~/turtlebot3_ws/src
```

Qué hace: Crea la carpeta del espacio de trabajo (workspace) y su subcarpeta `src`, donde irán los paquetes.

- `mkdir` → crea carpetas
- `-p` → crea toda la ruta de una vez y no da error si la carpeta ya existe
- `~/turtlebot3_ws/src` → la ruta del workspace y su carpeta de código fuente

1.2 Entrar a la carpeta `src`

```
cd ~/turtlebot3_ws/src/
```

Qué hace: Entra a la carpeta `src`, que es donde se clonan los paquetes.

- `cd` → cambia de carpeta (change directory)
- `~/turtlebot3_ws/src/` → la carpeta de destino

1.3 Descargar el paquete de simulaciones

```
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
```

Qué hace: Descarga desde GitHub el código del paquete de simulaciones del TurtleBot3.

- `git clone` → copia un repositorio de GitHub a tu PC
- `-b humble` → elige la rama “humble” (la versión para ROS 2 Humble)
- `https://...turtlebot3_simulations.git` → la dirección del repositorio que se descarga

1.4 Compilar el workspace

```
cd ~/turtlebot3_ws && colcon build --symlink-install
```

Qué hace: Vuelve a la raíz del workspace y compila todos los paquetes para que ROS pueda usarlos.

- `cd ~/turtlebot3_ws` → vuelve a la raíz del workspace
- `&&` → encadena: si el `cd` funciona, ejecuta lo siguiente
- `colcon build` → compila todos los paquetes del workspace
- `--symlink-install` → usa enlaces en vez de copias, así los cambios en archivos se reflejan sin recompilar

1.5 Abrir Gazebo y RViz juntos

```
ros2 launch turtlebot3_manipulation_gazebo gazebo.launch.py start_rviz:=true
```

Qué hace: Abre Gazebo y RViz al mismo tiempo, con el robot y su brazo cargados.

- `ros2 launch turtlebot3_manipulation_gazebo gazebo.launch.py` → abre la simulación del robot con brazo
- `start_rviz:=true` → además de Gazebo, abre también RViz

1.6 Activar el servo de movimiento

► Terminal 2

```
ros2 launch turtlebot3_manipulation_moveit_config servo.launch.py
```

Qué hace: Activa el servidor de servo de MoveIt (el “coso de movimiento”), que es lo que permite mover el brazo en tiempo real.

- `ros2 launch ... servo.launch.py` → arranca el nodo de servo que habilita el control del brazo

1.7 Abrir el teleoperador del brazo

► Terminal 3

```
ros2 run turtlebot3_manipulation_teleop turtlebot3_manipulation_teleop
```

Qué hace: Abre el teleoperador normal y el teleoperador de la garra: con él mueves la base y el brazo por teclado.

- `ros2 run ... turtlebot3_manipulation_teleop` → ejecuta el teleoperador específico del robot con brazo

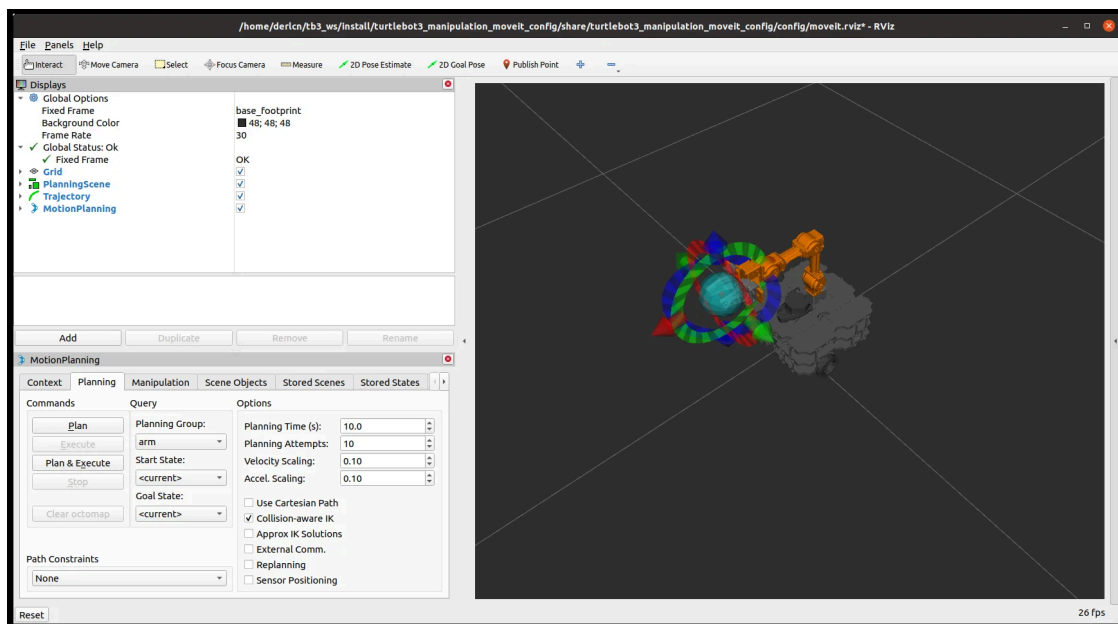
1.8 Planificar movimientos con MoveIt

► Terminal 4

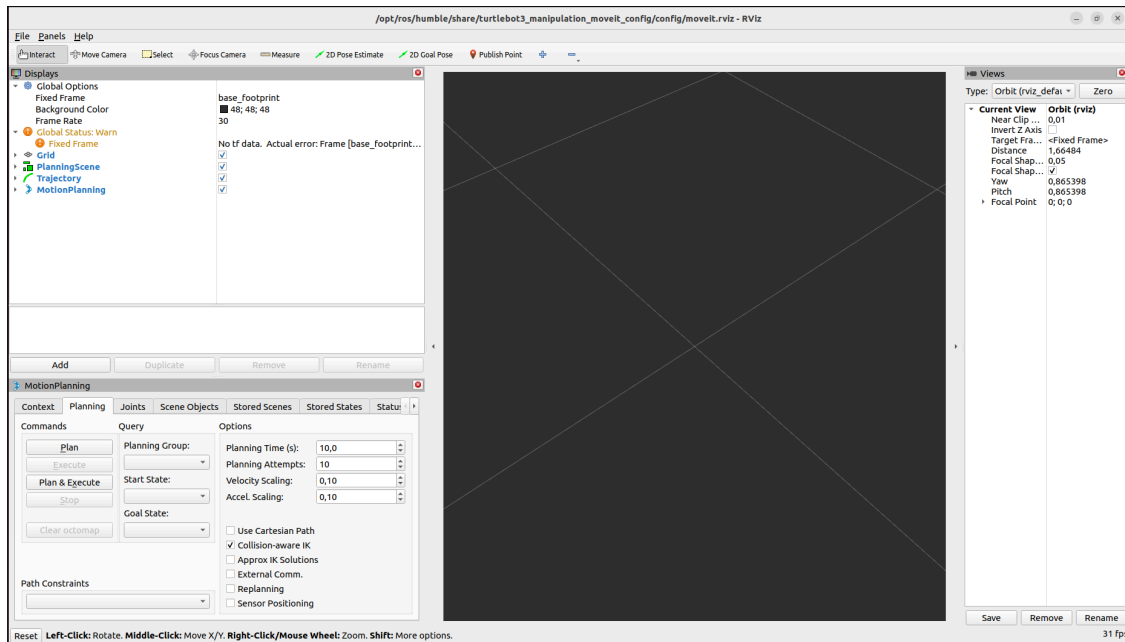
```
ros2 launch turtlebot3_manipulation_moveit_config moveit_gazebo.launch.py
```

Qué hace: Te deja planificar el movimiento antes de ejecutarlo (aunque no aparecerá el robot).

- `ros2 launch ... moveit_gazebo.launch.py` → lanza MoveIt junto con Gazebo para planificar trayectorias del brazo



RViz con MoveIt funcionando: el brazo y el marcador interactivo aparecen correctamente.



RViz sin datos ("No tf data"): la escena aparece vacía cuando falla la configuración.

Nota: No me funcionó así; solo me funciona si lo aplico de la siguiente manera:

Parche 1 Lanzar Gazebo

► Terminal 1

```
source ~/turtlebot3_ws/install/setup.bash
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_manipulation_gazebo gazebo.launch.py
```

Qué hace: Carga el workspace, define el modelo y abre Gazebo.

- `source ~/turtlebot3_ws/install/setup.bash` → carga tu workspace en la terminal para que reconozca tus paquetes
- `export TURTLEBOT3_MODEL=waffle_pi` → define el modelo del robot
- `ros2 launch ... gazebo.launch.py` → abre la simulación en Gazebo

Parche 2 Lanzar RViz con el parche forzado

► Terminal 2

```
export LC_NUMERIC="en_US.UTF-8"
source ~/turtlebot3_ws/install/setup.bash
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_manipulation_moveit_config moveit_core.launch.py
use_sim_time:=True
```

Qué hace: Fuerza el formato numérico en inglés y lanza MoveIt usando el tiempo de simulación.

- `export LC_NUMERIC="en_US.UTF-8"` → fuerza el punto decimal (formato inglés) para evitar el error de la coma
- `ros2 launch ... moveit_core.launch.py` → lanza el núcleo de MoveIt
- `use_sim_time:=True` → usa el reloj de la simulación en lugar del reloj del PC

Analizando los mensajes que aparecían en la terminal de RViz aparece: `parameter {arm.kinematics_solver_timeout} is of type {double}, setting it to {string} is not allowed`. Lo que

significa que los decimales, cuando Linux está en español, se escriben con coma y en inglés con punto; lo que hace es que lo interpreta como un string (texto) y falla.

2. SLAM

2.1 Abrir la simulación en Gazebo

► Terminal 1

```
ros2 launch turtlebot3_manipulation_gazebo gazebo.launch.py
```

Qué hace: Abre la simulación del robot con brazo en Gazebo.

2.2 Arrancar Cartographer

► Terminal 2

```
ros2 launch turtlebot3_manipulation_cartographer cartographer.launch.py
```

Qué hace: Arranca Cartographer para ir construyendo el mapa mientras el robot se mueve.

- `ros2 launch ... cartographer.launch.py` → lanza el nodo de SLAM que genera el mapa en tiempo real

2.3 Activar el servo

► Terminal 3

```
ros2 launch turtlebot3_manipulation_moveit_config servo.launch.py
```

Qué hace: Activa el servo para poder controlar el robot.

2.4 Abrir el teleoperador

► Terminal 4

```
ros2 run turtlebot3_manipulation_teleop turtlebot3_manipulation_teleop
```

Qué hace: Abre el teleoperador para conducir el robot y así recorrer y mapear el entorno.

Nota: Aunque la guía diga que se maneja el TurtleBot con O, K, L, ; en realidad es con i, j, k, l y se para con la barra espaciadora.

2.5 Guardar el mapa

► Terminal 5

```
ros2 run nav2_map_server map_saver_cli -f ~/map
```

Qué hace: Guarda el mapa que acabas de crear en dos archivos (`~/map.yaml` y `~/map.pgm`).

- `ros2 run nav2_map_server map_saver_cli` → ejecuta la herramienta que guarda el mapa actual
- `-f ~/map` → nombre y ruta base de los archivos de salida

3. Navegación

3.1 Abrir la simulación en Gazebo

► Terminal 1

```
ros2 launch turtlebot3_manipulation_gazebo gazebo.launch.py
```

Qué hace: Abre la simulación del robot con brazo en Gazebo.

3.2 Lanzar Nav2 con el mapa guardado

► Terminal 2

```
ros2 launch turtlebot3_manipulation_navigation2 navigation2.launch.py  
map_yaml_file:=$HOME/map.yaml
```

Qué hace: Arranca Nav2 con el robot y carga el mapa que guardaste en la sección de SLAM.

- `ros2 launch ... navigation2.launch.py` → lanza el sistema de navegación Nav2
- `map_yaml_file:=$HOME/map.yaml` → ruta del mapa que se cargará (el que guardaste antes)

4. Home Service Challenge

4.1 Instalar los paquetes del Home Service Challenge

► Terminal 1

```
cd ~/turtlebot3_ws/src/  
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_home_service_challenge.git  
cd ~/turtlebot3_ws && colcon build --symlink-install  
source ~/turtlebot3_ws/install/setup.bash
```

Qué hace: Descarga, compila y carga el paquete del reto Home Service Challenge.

- `git clone -b humble ...home_service_challenge.git` → descarga el paquete del reto desde GitHub
- `colcon build --symlink-install` → compila el paquete dentro del workspace
- `source ~/turtlebot3_ws/install/setup.bash` → carga el workspace para que la terminal reconozca el paquete nuevo

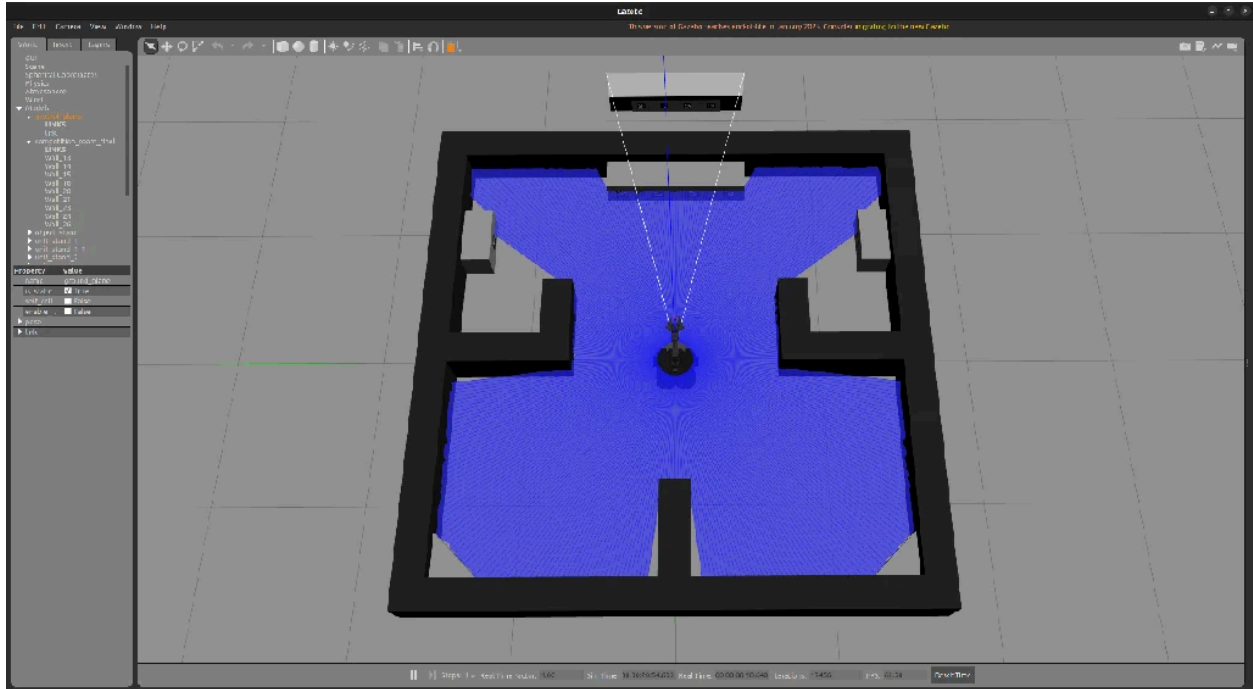
4.2 Correr la simulación de Gazebo

► Terminal 2

```
source ~/turtlebot3_ws/install/setup.bash  
ros2 launch turtlebot3_manipulation_gazebo turtlebot3_home_service_challenge.launch.py
```

Qué hace: Abre el escenario del reto en Gazebo (con el robot y los objetos a recoger).

- `ros2 launch ... turtlebot3_home_service_challenge.launch.py` → carga el mundo del reto en Gazebo



4.3 Ejecutar Nav2 y fijar la pose 2D en RViz

► Terminal 3

```
source ~/turtlebot3_ws/install/setup.bash
ros2 launch turtlebot3_home_service_challenge_tools navigation2.launch.py
```

Qué hace: Arranca Nav2 junto con RViz para que el robot pueda planear rutas.

Nota: Hacer la estimación de la pose inicial (botón 2D Pose Estimate en RViz). Sin esto el robot “no sabe dónde está” y la navegación falla.

4.4 Ejecutar el paquete principal (core)

► Terminal 4

```
source ~/turtlebot3_ws/install/setup.bash
ros2 launch turtlebot3_home_service_challenge_core core_node.launch.py
```

Qué hace: Lanza el cerebro del reto: detección de marcadores ArUco, estacionamiento frente a ellos y control del brazo.

- `ros2 launch ... core_node.launch.py` → arranca el nodo central que coordina toda la misión

4.6 Mandar órdenes al brazo

► Terminal 5

De forma individual — estacionar frente a un marcador:

```
ros2 topic pub -1 /target_marker_id std_msgs/msg/Int32 "{data: 0}"
```

Qué hace: Le ordena al robot estacionarse frente al marcador ArUco cuyo ID indiqués.

- `ros2 topic pub` → publica un mensaje en un topic
- `-1` → publica el mensaje una sola vez (no en bucle)
- `/target_marker_id` → el topic donde se envía el ID del marcador objetivo
- `std_msgs/msg/Int32` → el tipo de mensaje: un número entero
- `"{data: 0}"` → el contenido: el número del marcador (del 0 al 7)

Nota: Al usar este comando, asegúrese de incluir uno de los ID de los marcadores ArUco de un archivo `scenario.yaml`. En el mapa proporcionado existen los ID del 0 al 7. Para información detallada sobre el escenario, consulte la descripción de la configuración (cambiando `$MARKER_ID` por un número del 0 al 7).

Controlar el brazo para recoger o soltar un objeto:

```
ros2 topic pub -1 /manipulator_control std_msgs/msg/String "{data: 'pick_target'}"
ros2 topic pub -1 /manipulator_control std_msgs/msg/String "{data: 'place_target'}"
```

Qué hace: Ordena al brazo recoger (pick) o soltar (place) un objeto.

- `/manipulator_control` → el topic que da órdenes al brazo
- `std_msgs/msg/String` → el tipo de mensaje: texto
- `"{data: 'pick_target'}"` → orden de recoger el objeto
- `"{data: 'place_target'}"` → orden de soltar el objeto

Escenario completo (el robot hace toda la secuencia solo: ir, recoger, llevar, soltar y volver):

```
ros2 topic pub -1 /scenario_selection std_msgs/msg/String "{data: 'room1'}"
```

Qué hace: Ejecuta un escenario completo de forma automática, de principio a fin.

- `/scenario_selection` → el topic que lanza un escenario completo
- `"{data: 'room1'}"` → el nombre del escenario a ejecutar

Resultados

Al final cumplimos con el objetivo original, que es utilizar Nav2 para moverse alrededor del mapa de la casa, y tenemos la garra del TurtleBot funcionando, pero solo teleoperada; el proyecto falla al intentar automatizar este proceso.

Bibliografía

<https://docs.robotis.com/docs/systems/turtlebot3/overview/>

<https://github.com/autonomy-and-verification-uol/gwendolen-ros-turtlebot3/tree/master/maps>

Anthropic. (2026). Claude (Opus 4.8) [Large language model]. <https://claude.ai> Google. (2026).

Gemini [Large language model]. <https://gemini.google.com>

<https://docs.ros.org/en/humble/index.html>

<https://gazebo-sim.org/docs/fortress/getstarted/>